

MOESI-D: Delegating Contended Atomic Additions to the Home Cache Bank

Manav Sharma
Independent Researcher

Yaseen Alkhameri
Independent Researcher

Abstract—Many programs keep shared counters that every core keeps adding to: statistics, histograms, sketches for network telemetry, and the scatter-add steps of sparse and graph kernels. Each update changes only a few bytes, but today’s cache-coherence protocols move the entire cache line to whichever core is updating it. When many cores update the same lines, the line spends its time traveling instead of being used.

We introduce MOESI-D, which adds a Delegated (D) state to the MOESI protocol. Instead of moving the line to the update, MOESI-D moves the update to the line: a core sends a small message saying “add 1 here,” and the line’s home cache bank applies it with a simple adder and replies. Lines that are being fought over stay at the home bank; lines that one core uses on its own are left alone, because a small table at the home bank watches how often each line changes hands and only delegates the ones that change hands on nearly every update. Every update still completes in order, so programs see exactly the memory behavior they see today.

We implemented MOESI-D in `gem5` and checked every run for correctness. On x86’s standard memory ordering, MOESI-D is $1.26\times$ faster on many contended lines at 4, 8, and 16 cores, $1.6\text{--}1.7\times$ faster on a single hot counter and on a counter mixed with reads at 16 cores, and within 1.5% on workloads where cores rarely share lines. When software uses relaxed atomics, the many-line speedup reaches $3.3\times$ at 4 and 8 cores and $4.95\times$ on a 16-core mesh, with $7\times$ less on-chip traffic. The hardware cost is one adder and about 6 KB of state per L2 bank.

Index Terms—Cache coherence, atomic memory operations, hardware delegation, memory consistency, `gem5`, multi-core architectures.

I. INTRODUCTION

Cache coherence is optimized for read-sharing and single-writer locality. It is ill-suited for a narrower but pervasive access pattern: **many cores updating the same lines with small atomic read-modify-writes**. Examples include histogram and Count-Min-Sketch accumulation, statistics and abuse counters, telemetry aggregation, and the scatter-add phases of sparse-matrix and graph kernels. Our prior `gem5` study of MI, MESI, and MOESI on SpMV and GAPBS BFS [17] showed how much coherence traffic these workloads generate (e.g., HB/685_bus at 4 cores: 12.69M GETX and 12.27M FWD_GETX under MI) and proposed, conceptually, a Delegated state in which updates travel to the data instead of the data traveling to each update. This paper implements that state, measures it under the ordering

rules of deployed x86 hardware, and corrects an earlier version of this work (Section VII).

Two structural properties make line migration unnecessary for these updates:

- 1) **Commutativity and Associativity:** For integer addition, applying an update wherever the line currently resides, in any order, produces the same architectural state as migrating the line first.
- 2) **No-Return Semantics:** The issuing thread does not consume the previous value (e.g., x86 LOCK ADD vs. XADD, RISC-V AMOADD with `rd=x0`, ARMv8.1 STADD). The update can therefore be applied wherever the line already is.

MOESI-D exploits both. A no-return addition to a contended line is sent to the line’s **home L2 bank** as a compact 32-byte message, applied there by a small executor, and acknowledged with an 8-byte reply. The line can be **retained** at the home in state D, and reads of a retained line are served from a snapshot without moving it. Cache-line migration disappears for the lines that need it most.

Two questions decide whether this is a win or a loss, and most of this paper is about answering them correctly:

- **Ordering.** Under x86 Total Store Order (TSO), a younger store may not become visible before an older atomic has been applied. A delegated add therefore stays outstanding until the home acknowledges it. MOESI-D recovers the batching a local owner enjoys with a TSO-legal **same-line batching** rule (Section III-F).
- **Admission.** Delegation wins on lines that change hands on nearly every update and loses on lines the owner would have kept. A **change-rate admission gate** at the home bank tells the two apart by counting ownership changes per update, in both conventional and delegated mode, and serves its refusals as an ordinary ownership request in a single trip (Section III-G).

Importantly, D-state remains a strict performance layer: any delegated request can be refused before it is applied, after which the requester completes the operation through the conventional GETX path. No operation is ever applied twice or lost.

Contributions:

- A **Delegated coherence state** for no-return 32- and 64-bit integer addition, fully implemented in `gem5` Ruby/SLICC on MOESI_CMP_directory, with a bounded home-bank executor, combining at both requester and home, far

Implementation and evaluation on `gem5` v25.1.0.1. Artifact, every run configuration, and all raw statistics available at: <https://github.com/msharma2026/MOESI-D-Architecture>. This version supersedes v1.0.0; Section VII lists the corrections.

reads from retained lines, and a refusal path that guarantees forward progress.

- An **ordering-correct core interface**: a store-class AADD micro-op that completes only on acknowledgement, and a TSO-legal same-line batching rule verified by dedicated litmus tests.
- A **change-rate admission gate** that separates owner-friendly shared lines from migration-heavy ones, with cheap single-trip rejection, plus a record of the admission mechanisms that were built, measured, and rejected.
- One fixed configuration evaluated on **nine workloads at 4, 8, and 16 cores**, on crossbar and mesh networks, under both strict and relaxed ordering, with every run oracle-checked, and with latency-sensitivity and region-of-interest analyses that bound every headline number.

II. BACKGROUND AND MOTIVATION

A. MOESI and the Line-Migration Overhead

In directory-based MOESI, a write to a line held elsewhere sends a GETX to the home, which forwards it to the current owner; the owner ships the full line to the requester and invalidates its copy. When several cores keep adding to the same lines, nearly every add is such a migration. In our 4-core baseline, 512 contended lines see **one GETX per add** (4.1M of each). Not every shared line behaves this way: on a Count-Min sketch there is one GETX per 4.8 adds, and on a single hot counter one per 4.2, because the owner gets several local hits before losing the line. This ratio, how many updates an owner gets per ownership, is what decides whether delegation pays.

B. Memory Ordering

x86 is TSO: stores and atomic read-modify-writes become visible in program order. A core therefore cannot treat a delegated add as complete when its message leaves the core; doing so lets a younger store become visible first. The add must remain outstanding until applied, and the store queue holds younger stores behind it. On a relaxed model (ARMv8 or RISC-V with relaxed atomics, or x86 with an extension such as Intel’s RAO-INT), a no-return add may become visible after younger stores, provided software fences where it publishes. We report both and label every number.

C. Design Space of Prior Approaches

Table I places MOESI-D among prior techniques for contended updates.

Remote and memory-side atomics already execute updates away from the core. MOESI-D adds a **retained coherence state** with far reads, combining at both ends, and an **admission policy** that decides per line, at run time and from the line’s own ownership history, whether delegation or migration is cheaper.

III. THE DELEGATED (D) STATE

A. Core Mechanism

MOESI-D accelerates one operation: modular addition of a 32- or 64-bit integer to an aligned word, with no return value. An L1 issuing such an add to a line it does not own sends a

TABLE I
DESIGN SPACE OF HARDWARE SUPPORT FOR CONTENTED UPDATES

Strategy	Representative	Execution Site
Privatization	COUP [1], COMMTM [2]	Local core copy
In-Situ Atomics	Atomic Cache [15]	8T SRAM (GPU)
Asynch. Reduct.	Tascade [9]	NoC Routers
Combining Net.	NYU Ultracomputer [10]	Network switches
Coherence Queue	QOLB [16]	L1/L2 linked lists
Mem-side PIM	AMD US 11,726,918 [6]	DRAM PIM
Per-op AMO	DynAMO [4], Delegato [5]	Near-core vs. LLC
Remote Atomics	Intel RAO [7]	LLC / memory ctrl.
Softw. Combine	Flat Combining [8]	Single thread lock
Delegation	MOESI-D (This work)	Home L2 bank

DSTATE_REQ to the line’s home L2 bank. As illustrated in Fig. 1, the home either:

- **accepts**: obtains the line if needed (recalling it from an owning L1 or fetching it from memory through the ordinary paths), applies the add, and returns DSTATE_ACK; or
- **refuses** before applying anything, after which the requester completes the add through the conventional path.

After applying an add, the home either returns the line to its previous state or **retains it in D**. A line in D is held only at the home, its sole writer; no L1 holds a copy. If the requesting L1 already owns the line, the add is applied locally, exactly as on the conventional protocol.

B. Messages and Size Classes

A single-word DSTATE_REQ is **32 bytes** (operand, word offset, requester, tag). When the requester combines adds to several words of one line, a masked “delta line” is sent in a 48-, 80-, or 144-byte class. DSTATE_ACK and DSTATE_NACK are **8 bytes**; an ACK carrying the updated word is 16 bytes. A NACK always means the operation was not applied. It is never a timeout, and no operation is ever applied twice.

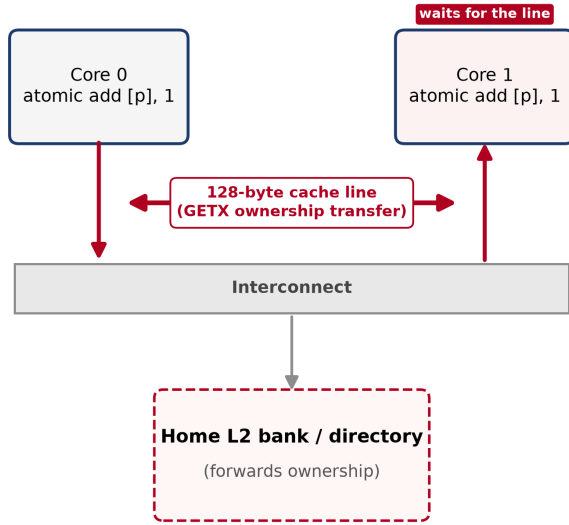
C. Home-Bank Executor

Each L2 bank incorporates a bounded executor: a **16-entry** queue of accepted operations, a 4-cycle initiation interval, and a 20-cycle service time. A 32-entry hot-word buffer with a 4-cycle hit time can be enabled for single-word workloads. When the executor is full, requests wait in the input buffer (back-pressure) instead of being refused, and a request for a line whose previous operation is still executing waits rather than being rejected. Transitions follow an *apply-then-ACK* discipline. The executor latencies are design assumptions; Section VI-D measures their effect.

D. Combining at Both Ends

At the requester, the Sequencer folds a consecutive run of adds to one line into a single request: same-word adds become one operand, different words become a masked delta line, up to 16 operations per request, all completed by its one ACK. At the home, an add arriving while an earlier accepted add

Conventional MOESI: line migrates on every update



MOESI-D: update moves to the home bank

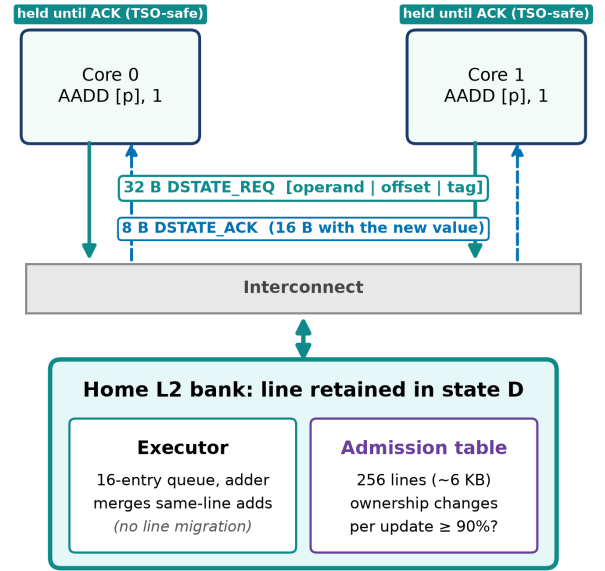


Fig. 1. Conventional Line Migration vs. MOESI-D Home-Bank Delegation. In conventional MOESI (left) every update by a different core moves the 128-byte line. In MOESI-D (right) the core sends a 32-byte request, the home L2 bank’s executor applies it in place and replies with an 8-byte acknowledgement, and the line stays at the home in state D. The admission table decides which lines are delegated. The issuing add stays outstanding until its acknowledgement, preserving TSO.

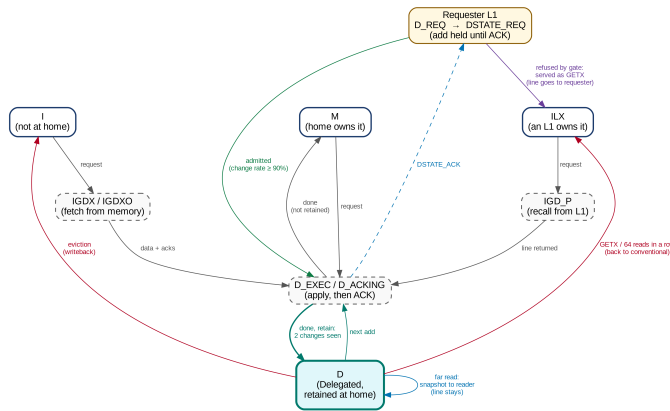


Fig. 2. The Delegated (D) State Machine at the Home L2 Bank. Admitted requests are applied in D_EXEC/D_ACKING and acknowledged; lines that show sustained ownership changes are retained in D, where further adds apply in place and reads are served as snapshots. A GETX, a long read run, or an eviction returns the line to conventional states. Requests the gate refuses are served as the requester’s GETX.

to the same line has not yet been applied is merged into it (up to 64), and each merged requester receives its own ACK. Because addition is associative and every combined operation completes together, combining is invisible to software. At 16 cores, home merging absorbs **3.5M of 4.1M** requests on the 512-line workload.

E. Far Reads

A read of a line retained in D is served by sending a snapshot to the reader, which uses it once and does not cache it; the line stays at the home. After 64 consecutive reads with no intervening update, the line is treated as read-mostly and the next read takes the ordinary path, demoting it. A GETX of a D line returns it to conventional ownership.

F. Ordering: AADD and Same-Line Batching

The operation is exposed through an explicit instruction, AADD (encoding 0F 38 FC /r, matching the AADD encoding in recent x86 documentation), which writes no flags. The upstream LOCK ADD macro-ops are left unchanged. AADD is a **store-class micro-op**: it occupies a store-queue entry, issues only after commit (so speculative adds never leave the core), and completes only when the memory system acknowledges it. Store-to-load forwarding from an AADD is disabled, since it has no local value to forward.

Under TSO the store queue allows one store in flight, so a run of delegated adds would be serialized at one round trip each, while a local owner would apply the same run back to back. MOESI-D allows a younger no-return add to leave the store queue while older ones are in flight **only if it targets the same line**; a store to any other line still waits for all of them. The Sequencer sends the run as one request and the home applies it as one operation, so no observer can see the adds out of order or see a later store before any of them. This is legal under TSO and verified by a dedicated litmus test (Section V). With software that uses relaxed atomics and fences, an optional

mode lets AADD leave the store queue without waiting for older stores; it is reported separately.

G. Admission: the Change-Rate Gate

Delegation pays when a line would otherwise migrate on nearly every update, and loses when the owner would have kept hitting the line locally. The home cannot see an owner's local hits, but it can count ownership changes.

Each L2 bank keeps a **256-entry admission table**. For each line it holds decayed counts of updates and of *changes*, where a change is an access by a core other than the line's previous writer. In conventional mode a change is a GETX from a non-owner; in delegated mode it is an update that would have migrated the line had it not been delegated. Counting in both modes keeps the estimate alive after delegation begins, when there are no migrations left to observe. Counts halve every 64 updates, so a steadily contested line keeps a steady ratio and never oscillates between modes; a line idle for 200,000 cycles starts over.

A request is admitted only when the line has shown at least two changes and **changes make up at least 90% of its recent updates**. A refusal is served as the requester's ordinary GETX: the line moves to the requester through the conventional path and the L1, still holding the operation, applies it on arrival, in **one trip instead of a NACK followed by a GETX**. Only the gate's own decision is served this way; a line retained in D and a structural refusal (line mid-transition, executor or TBEs full) still receive a NACK, because those refusals are temporary. The same table drives retention: a line is retained in D once it has shown two changes, so far reads engage on read-mixed lines immediately.

H. Deadlock Avoidance and Forward Progress

Requests, forwards, and responses use separate virtual networks with an acyclic dependency chain. A delegated request waiting at the home is woken by the transition that ends the state it waits on: every D transient reaches an execution completion, and every base transient reaches a transition that wakes waiters. Refusals are terminal, returning the operation to the requester's conventional path, so delegation can always be declined without affecting forward progress.

IV. IMPLEMENTATION

MOESI-D is constructed by extending gem5's Ruby MOESI_CMP_directory protocol. At the L2 we introduce stable state D and transient states for acquisition (IGDX, IGDXO, IGD_P), draining (DX_DRAIN), execution (D_EXEC_M, D_EXEC_D), and acknowledgement (D_ACKING_M, D_ACKING_D); at the L1, D_REQ, D_REQ_S, and D_RETRY. The global directory runs the unmodified MOESI_CMP_directory state machine; D is retention local to the home L2 bank. Executor admission, the hot-word buffer, masked addition, and the admission table are a C++ structure exposed to SLICC.

Outside the protocol, the gem5 changes are: the Sequencer (requester combining, remote completion of combined operations, completing a waiting load from the ACK value); the O3

load-store unit (store-class atomic, forwarding guard, same-line batching, relaxed option); x86 decode of AADD; network message size classes; functional-read support in Garnet; and a functional-read priority on the L2 and directory so syscall-emulation reads of a line in flight read the L2 copy or memory, as gem5's MESI protocols already do.

Hardware Overhead: Per L2 bank, MOESI-D adds a 32/64-bit integer adder with masked multi-word application, a 16-entry operation queue and merge record, an optional 32-entry hot-word buffer, and the 256-entry admission table (line tag, two small counters, last-writer ID, timestamp; **about 6 KB**). A 256-entry table produced results identical to a 1,024-entry table on every measured row. The core adds one store-class micro-op, a forwarding guard, and a line comparison in the store queue. **Nothing is added to the L1 beyond protocol states**. Physical-design synthesis remains outside the scope of this architectural study.

V. METHODOLOGY

Simulation Configuration: Simulations use gem5 v25.1.0.1 in syscall-emulation mode with X86O3CPU (and X86MinorCPU for correctness gates), Ruby, and the Garnet network: 128-byte lines, 32 kB L1 instruction and data caches, four L2 banks (`--l2_size=256kB, --num-l2caches=4`), and a crossbar. Mesh runs use a 4×4 Mesh_XY with 8 L2 banks and 8 directories. The 16-core runs use 256-entry message buffers, because gem5's deadlock detector fires on the conventional baseline at the default size.

Comparison: Every speedup compares MOESI-D with **the same binary and the same CPU options with D-state disabled**. A CPU-side option such as same-line batching therefore also applies to the baseline.

One Configuration: A single knob set is used for every workload: the executor of Section III, requester combining (16), home merging (64), far reads, masked delta lines, ACK carrying the value, the 90% change-rate gate with idle decay, a 256-entry table, single-trip refusal, and retention on two changes. The strict set adds same-line batching; the relaxed set adds relaxed no-return atomics. The hot-word buffer is used only for single-word workloads. Retained (persistent) mode is reported. Timing is whole-program `simSeconds` unless stated; Section VI-D reports region-of-interest timing.

Benchmarks:

- `scatter_multi`: each thread adds to 512 counters, each on its own line.
- `hot word`: all threads add to one 32-bit counter.
- `scatter`: 32 counters packed into one 128-byte line.
- `scatter_rw`: adds to a 32-counter line with a full-line read every eighth add.
- `cms`: 4×4096 Count-Min Sketch, Zipfian keys with a shifting hot set.
- `phased`: alternating update and read phases on 512 lines.
- `litmus`: Zipfian updates over 2,048 counters.
- `graph`: push-style scatter-add over a 65,536-vertex graph (memory-bound).

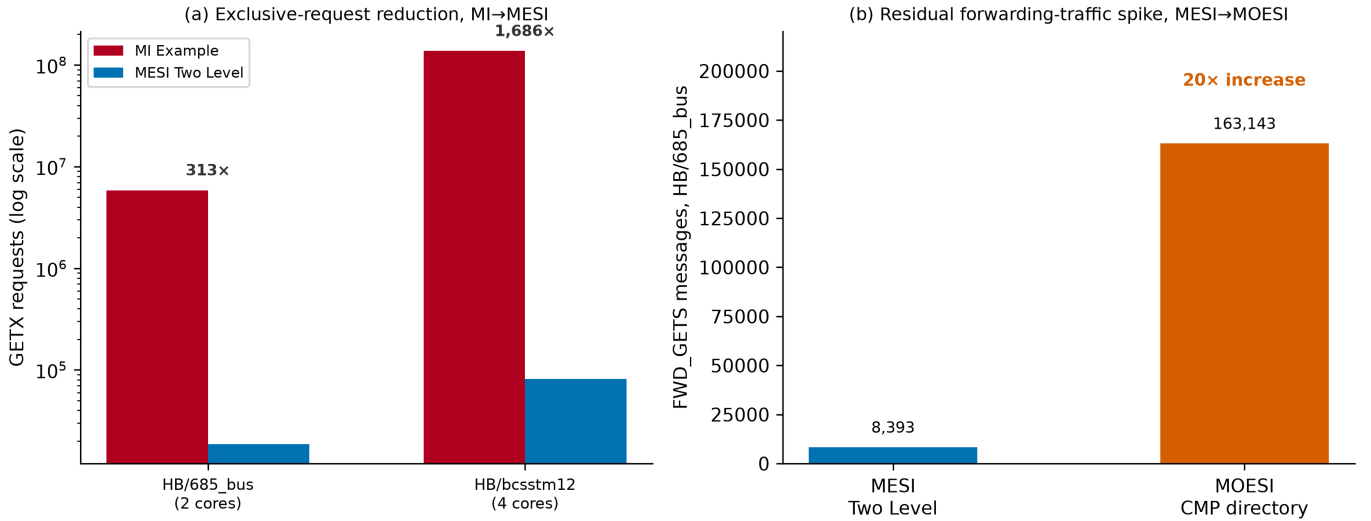


Fig. 3. Baseline Coherence-Traffic Characterization, with data adapted from our prior study [17]. (a) MESI reduces exclusive ownership requests compared to MI. (b) MOESI increases read-forwarding traffic (FWD_GETS) relative to MESI on HB/685_bus. These baseline measurements motivate the present work; they are not MOESI-D results.

Correctness: Every run checks its final sums against an oracle and passes a **coverage gate** requiring that the mode’s protocol path was actually exercised. For every mechanism and knob combination we additionally ran a coherence regression (mixed loads, stores, adds, and delegated adds with exact sums) in conventional, delegated, retained, and forced-refusal modes on O3 TSO, Minor TSO, and O3 fenced-relaxed; an ordering litmus test (no younger store observed before an older add); a same-line litmus test (two adds to one line, then a flag store to another; the flag may never be seen first); random stress at 4 and 16 cores including the `gem5.debug` binary; and engine unit tests covering 32/64-bit wraparound, every aligned offset, masked and merged arithmetic, and the admission table. All gates pass: every regression matrix passes in all four modes, and the litmus tests show **0 forbidden outcomes in over 240,000 rounds**.

VI. EVALUATION

A. Scaling Under x86 Ordering

Table II and Fig. 4 report MOESI-D under TSO, the ordering of deployed x86 software.

TABLE II
SPEEDUP OVER CONVENTIONAL MOESI, x86-TSO

Workload	4 cores	8 cores	16 cores
512 contended lines	1.26×	1.26×	1.26×
One hot word	1.00×	1.01×	1.62×
32 counters, one line	1.00×	—	—
Read every 8 adds	0.74×	1.11×	1.72×
Count-Min sketch	0.99×	0.99×	1.00×
Phased update/read	1.26×	—	—
Zipfian counters	1.00×	—	1.00×
Graph push	1.00×	—	1.00×

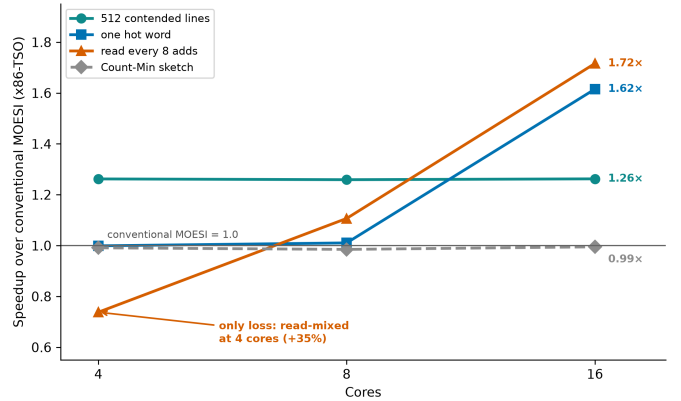


Fig. 4. Speedup vs. Core Count under x86-TSO. Many contended lines win at every core count; a single hot counter and a read-mixed counter cross over and reach 1.62× and 1.72× at 16 cores; the Count-Min sketch, where owners keep their lines, stays at parity. The only loss is the read-mixed counter at 4 cores.

On **many contended lines**, MOESI-D is 1.26× faster at every core count: each update is one 32-byte request instead of a line migration. On a **single hot counter** at 4 cores the owner batches its own adds, and same-line batching lets delegation match it exactly; at 16 cores an owner rarely keeps the line long enough to batch, and delegation wins by **1.62×**. The **read-mixed counter** is the one case MOESI-D loses (0.74× at 4 cores): under TSO the read behind each add waits for the home’s acknowledgement, while the owner completes both locally in a few cycles. From 8 cores the owner no longer keeps the line, and the same workload becomes a **1.11×** win at 8 cores and **1.72×** at 16. On the **Count-Min sketch**, **Zipfian counters**, and **graph push**, the gate keeps owner-friendly and uncontended lines on the conventional path, and MOESI-D

stays within 1.5% of the baseline.

B. Relaxed Ordering

When software uses relaxed no-return atomics with explicit fences, delegated adds no longer wait for one another, and the many-line case scales far beyond the strict result (Fig. 5).

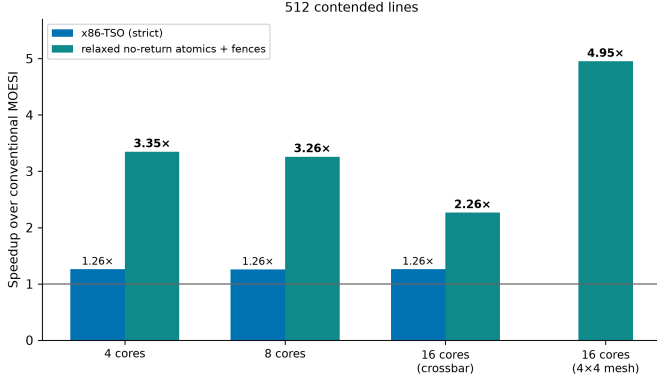


Fig. 5. Strict vs. Relaxed Ordering on 512 Contended Lines. Under TSO, MOESI-D is 1.26 $\times$ faster at every core count. With relaxed atomics and fences it reaches 3.35 $\times$ (4 cores), 3.26 $\times$ (8 cores), 2.26 $\times$ (16 cores, crossbar), and 4.95 $\times$ on a 16-core 4 $\times$ 4 mesh, where the baseline’s migrations cross more hops.

The phased update/read workload reaches **3.01 $\times$** at 4 cores and **2.29 $\times$** at 16 under relaxed ordering. The single hot word, the Count-Min sketch, and graph push remain at parity; the read-mixed counter at 4 cores remains a loss (0.68 $\times$). These numbers require software to use relaxed atomics, or an ISA extension such as RAO-INT; deployed x86 software obtains the strict results above.

C. Admission: Telling Shared Lines Apart

Fig. 6 shows why admission is decided by ownership changes rather than by the number of writers. The Count-Min sketch and the 512-line scatter both have exactly four writers per line at 4 cores, yet the owner wins the first and loses the second. A distinct-writer gate cannot separate them: set to refuse the sketch, it also refuses the 512 lines, and every refusal then costs a NACK round trip before the migration. The change-rate gate sees the difference directly, because the sketch’s lines change hands once every 4.8 updates and the scattered lines on every update. With single-trip refusal, a strict 90% threshold costs nothing on the lines that should be delegated, and the sketch goes from **0.80 $\times$ to parity**. At 16 cores, where every line changes hands on nearly every update, the gate admits everything and the full speedup is kept.

D. Sensitivity to Home Latency and Timing Window

The executor’s 20-cycle service time is an assumption, so Fig. 7(a) varies it. The read-mixed loss at 4 cores changes by only about five points across a 3 $\times$ range: it is bounded by the network round trip under TSO, not by the executor. The many-line case is the opposite. It is executor-bound, and a 10-cycle executor would raise its 4-core speedup from 1.26 $\times$

to **1.65 $\times$** . Varying the hot-word buffer’s hit time (2–8 cycles) leaves the hot word at parity and does not help the read-mixed counter.

Whole-program timing includes thread start-up and exit. Fig. 7(b) compares it with region-of-interest timing (libm5 markers around the measured loop). The single hot word at 16 cores is **2.94 $\times$** inside the loop versus 1.62 $\times$ whole-program, so we quote it as **1.6–2.9 $\times$** . The 512-line relaxed result is 1.88 $\times$ inside the loop versus 2.26 $\times$, because the baseline’s start-up migrates every line sixteen ways. The read-mixed result agrees (1.75 $\times$ vs. 1.72 $\times$).

E. On-Chip Traffic

Fig. 8 reports injected flits measured with Garnet. On the many-line workloads, traffic falls to **0.13 $\times$** of the baseline at 4 cores and **0.15–0.16 $\times$** at 16 cores on both crossbar and mesh. On single-line workloads the effect reverses: traffic rises to 1.73 $\times$ (read-mixed, 4 cores) and 2.76–2.85 $\times$ (hot word and read-mixed, 16 cores), where MOESI-D is nevertheless 1.6–1.7 $\times$ faster. Flit counts are an interconnect dynamic-energy proxy only.

F. Why the Remaining Loss Remains

On the read-mixed counter at 4 cores, the owner completes an add and the following read locally in a few cycles; with delegation, TSO makes the read wait for the home’s acknowledgement. Carrying the updated word in the ACK satisfies a waiting read only when it reads that word, and this workload reads the whole line. We analyzed and measured three further remedies and rejected each. A full-line ACK forfeits the traffic saving that justifies delegation. Proactive snapshot pushes must be invalidated before every remote add. Speculative forwarding of local-copy-plus-delta, verified by the ACK, was evaluated with an oracle in the Sequencer: the prediction would have been right **43%** of the time on the read-mixed counter at 4 cores, **6%** at 16, and **0%** (of 539,510) on the hot word, because another core’s update almost always reaches the home first.

G. Mechanisms Built and Rejected

We implemented, measured, and removed four admission mechanisms; their results constrain the design.

- **Distinct-writer gate:** cannot separate owner-friendly shared lines from migration-heavy ones (Fig. 6), and its epoch resets cut the 16-core win from 2.26 $\times$ to 1.79 $\times$.
- **Requester-side owner-tenure predictor:** the L1 records how many adds it performed per ownership. It made the 4-core sketch a tie, but at 16 cores an owner gets two or three adds in before the recall, so it read “owner wins” where the owner loses, turned the hot word from 1.6 $\times$ faster into 18% slower, and required a 1,024-entry table per L1.
- **Adaptive gate threshold:** no measurable benefit.
- **Serving every refusal as a GETX:** turned temporary refusals at a busy home into migrations, taking the 16-core read-mixed counter from 1.71 $\times$ faster to 44% slower; restricted to the gate’s own decisions.

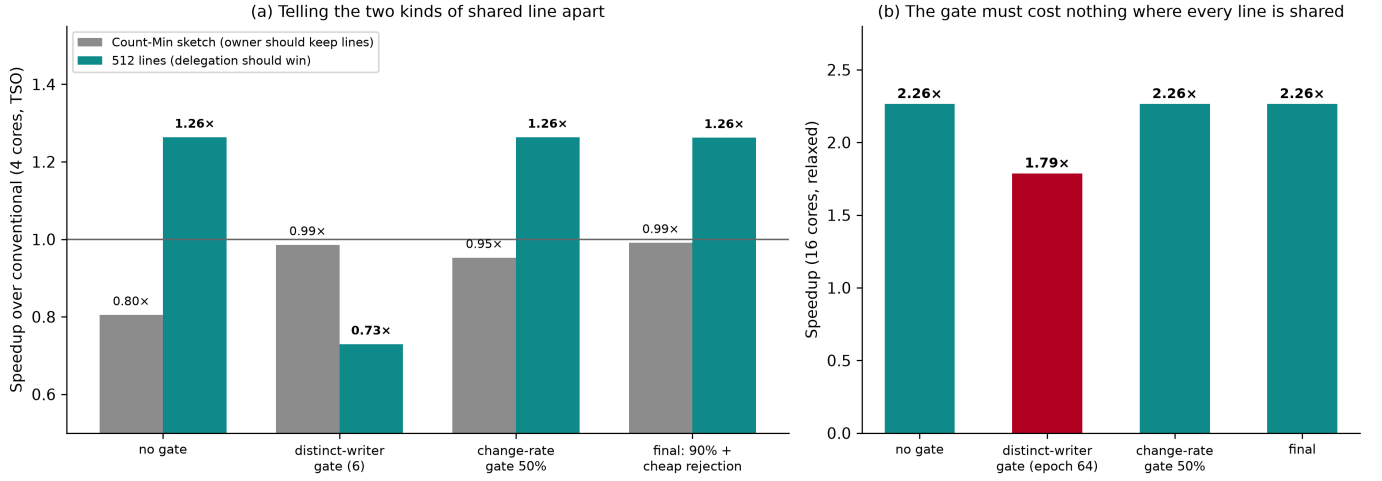


Fig. 6. Admission Policy. (a) At 4 cores under TSO, the Count-Min sketch (owners should keep their lines) and 512 scattered lines (delegation should win) both have four writers per line. Without a gate the sketch loses 20%; a distinct-writer gate set to refuse the sketch also refuses the 512 lines (0.73 $\times$, remote mode); the change-rate gate separates them, and with a 90% threshold and single-trip refusal the sketch is at parity while the 512 lines keep 1.26 $\times$. (b) At 16 cores every line is contended, and the gate must cost nothing: the distinct-writer gate’s epoch resets cut the win to 1.79 $\times$, while the change-rate gate keeps the full 2.26 $\times$.

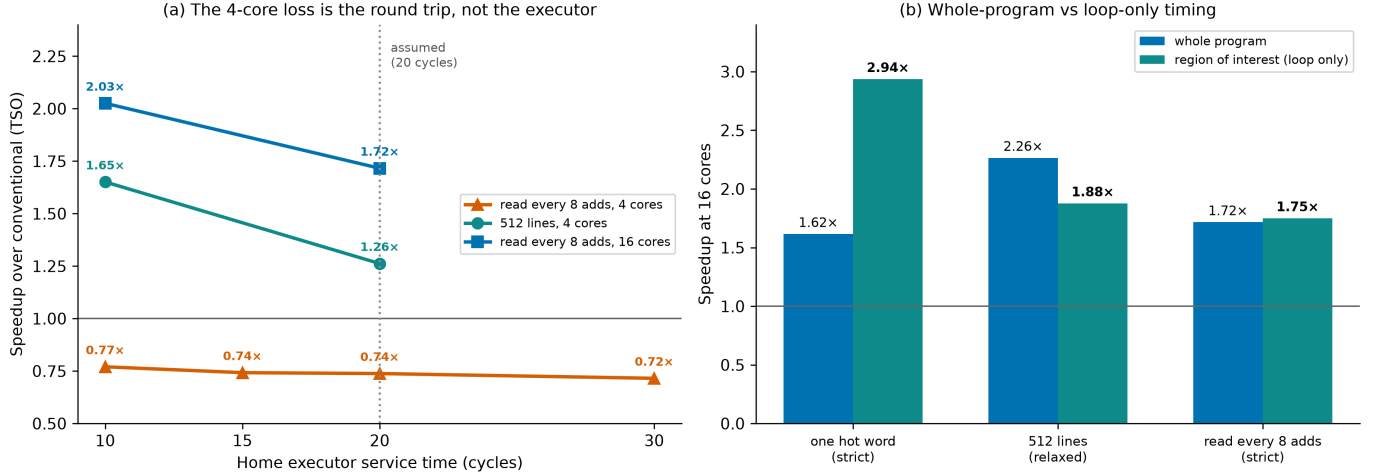


Fig. 7. (a) Sensitivity to the home executor’s service time. The read-mixed loss at 4 cores moves only from 0.77 $\times$ to 0.72 $\times$ across 10–30 cycles; it is the network round trip, not the executor. The many-line case is executor-bound and would rise from 1.26 $\times$ to 1.65 $\times$ with a 10-cycle executor; read-mixed at 16 cores from 1.72 $\times$ to 2.03 $\times$. (b) Whole-program vs. loop-only timing at 16 cores. The hot-word speedup is 2.94 $\times$ inside the loop (thread start-up dominates the short run); the 512-line relaxed speedup is 1.88 $\times$ (part of the whole-program win is the baseline’s slower start); read-mixed agrees.

In-network combining was not built: on the crossbar a request makes one hop, and home merging already absorbs most requests.

VII. CORRECTIONS TO THE EARLIER VERSION

The earlier version of this work (artifact v1.0.0) made claims that do not hold for the ordering-correct implementation evaluated here:

- 1) **Early completion.** It completed the atomic when the request left the L1, which allows a younger store to become visible before the add is applied and violates TSO. The operation now stays outstanding until acknowledged. The “fire-and-forget” speedups (approximately 2 $\times$ on out-of-order cores, 2.10–2.75 $\times$ on *scatter*) are withdrawn.

- 2) **Software transparency.** It redirected ordinary x86 LOCK ADD. LOCK ADD is now unchanged; the explicit AADD instruction is used.
- 3) **Operators.** Only 32- and 64-bit integer addition is implemented, not min/max, bitwise, or floating-point reductions.
- 4) **Execution site.** Delegated operations execute at the home L2 bank, not at a designated sole-writer core, and the directory state machine is unmodified.
- 5) **Headline numbers.** The 99.9% reduction in line migrations was measured on one in-order configuration and is not a general property; the 2.1 $\times$ flit reduction was measured on a single-line workload, where the current

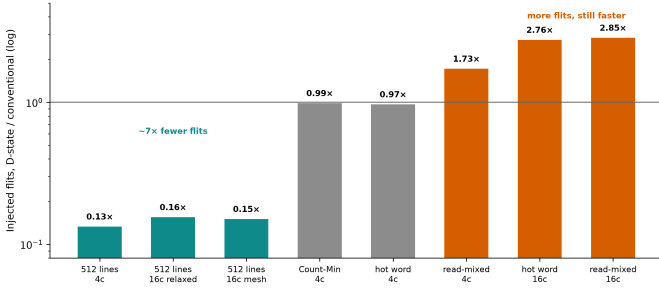


Fig. 8. Injected Network Flits Relative to Conventional MOESI. On many contended lines, 32-byte requests and 8-byte acknowledgements replace 128-byte migrations and cut traffic about 7 $\times$. On single-line workloads, the owner would have kept the line, every delegated add is a message, and traffic rises 1.7–2.9 $\times$, even where MOESI-D is faster.

design increases flits; the 5.3 $\times$ working-set speedup was not reproduced by the ordering-correct implementation. These are withdrawn and replaced by Section VI.

VIII. DISCUSSION AND LIMITATIONS

- **Workload Scope:** All results are from microbenchmarks and kernels in syscall-emulation mode at 4–16 cores. The application-shaped kernels (graph push here, SpMV in earlier rounds) are at parity. Evaluation on production workloads with contended counters at 32–64 cores remains future work. Earlier relaxed-ordering mesh results at 32 and 64 cores (4.87 $\times$ and 3.23 $\times$) predate the final admission policy and were not rerun.
- **Remote-Atomics Baseline:** We do not compare against a home-side atomic design without retention, far reads, or admission (an RAO-INT-style design); that comparison would isolate what the D state contributes beyond remote execution.
- **Consistency Model:** The largest speedups require relaxed atomics in software or an ISA extension; deployed x86 software obtains the strict results.
- **Executor Latency:** Service times are assumed; Section VI-D bounds their effect, and a circuit-level estimate is the largest remaining lever on the many-line results.
- **Timing Window:** Runs are deterministic and single; whole-program timing includes thread start-up and exit, with region-of-interest results reported where they differ.
- **Power and Area:** Flit counts are reported as an interconnect dynamic-energy proxy; area and package-level energy (e.g., via McPAT) remain future work.

IX. RELATED WORK

Prior SpMV and Graph Coherence Study: Our prior study [17] compared MI, MESI, and MOESI in gem5 on SpMV and GAPBS BFS and proposed a conceptual Delegated state in which updates are sent to a sole writer instead of migrating the line. It did not implement or evaluate that state; the present work does.

Cache-Level Commutativity: COUP [1] and COMMTM [2] replicate lines across cores and reduce at conflict or read time.

MOESI-D keeps one stationary copy at the home and needs no reduction pass.

Remote and Memory-Side Atomics: Intel RAO [7], AMD’s memory-local coalescing [6], and GPU L2 atomics execute updates at the LLC or memory controller. MOESI-D also executes at the home cache bank; its additions are a retained coherence state with far reads, two-sided combining, and per-line admission from ownership history.

In-Cache and SRAM Atomics: Atomic Cache [15] modifies 8T SRAM arrays for in-situ GPU atomics. MOESI-D uses standard SRAM and a small adder beside the cache controller.

Dynamic AMO Placement: DynAMO [4] and Delegato [5] choose near-core or LLC execution per operation. MOESI-D chooses per line, from the line’s measured ownership-change rate.

Combining and Reduction Networks: The NYU Ultra-computer [10] combined fetch-and-add in network switches, and Tascade [9] builds asynchronous reduction trees with specialized caches. MOESI-D combines at the requester and at the home bank and needs no changes to the network.

Synchronization in Coherence and Software: QOLB [16] integrated queue-based locking into coherence; flat combining [8] serializes updates through a software combiner. MOESI-D performs the equivalent combining in hardware, without a lock.

X. CONCLUSION

MOESI-D shows that contended no-return atomic additions can be resolved by moving the update to the line instead of the line to the update, without changing the memory ordering software relies on. A Delegated state at the home cache bank, combining at both ends, and admission by the measured ownership-change rate make MOESI-D **1.26 $\times$ faster on many contended lines at every measured core count** and **1.6–1.7 $\times$ faster on contended single lines at 16 cores** under x86-TSO, while staying within 1.5% where lines are not contended. With relaxed atomics, the many-line speedup reaches **3.3 $\times$** , and **4.95 $\times$** on a 16-core mesh, with about **7 $\times$ less** on-chip traffic. The design needs one adder and about 6 KB of state per L2 bank and nothing in the L1 beyond protocol states.

REFERENCES

- [1] G. Zhang, W. Horn, and D. Sanchez, “Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems (COUP),” in *Proc. 48th Int. Symp. Microarchitecture (MICRO)*, 2015, pp. 13–25.
- [2] G. Zhang and D. Sanchez, “Leveraging hardware caches for memoization to exploit semantic commutativity in hardware speculation (COMMTM),” in *Proc. 49th Int. Symp. Microarchitecture (MICRO)*, 2016, pp. 1–13.
- [3] W. H. Horn, “Cache-coherent commutative operations,” M.S. thesis, Massachusetts Institute of Technology, Cambridge, MA, 2015.
- [4] R. Soria-Pardos, A. Armejach, D. Suárez-Gracia, et al., “DynAMO: Improving parallelism through dynamic placement of atomic memory operations,” in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, 2023, pp. 1–14.
- [5] R. Soria-Pardos et al., “Delegato: Locality-aware atomic memory operations on chiplets,” in *Proc. 58th Int. Symp. Microarchitecture (MICRO)*, 2025.
- [6] Advanced Micro Devices, Inc., “Dynamically coalescing atomic memory operations for memory-local computing,” U.S. Patent 11,726,918 B2, Aug. 15, 2023.

- [7] Intel Corporation, “Remote atomic operations in multi-socket systems,” U.S. Patent 10,296,459, May 21, 2019.
- [8] D. Hendler, I. Incze, N. Shavit, and M. Tzafrir, “Flat combining and the synchronization-parallelism tradeoff,” in *Proc. 22nd ACM Symp. Parallelism in Algorithms and Architectures (SPAA)*, 2010, pp. 355–364.
- [9] M. Sutherland et al., “Tascade: Hardware support for atomic-free, asynchronous and efficient reduction trees,” *arXiv:2311.15810*, 2023.
- [10] A. Gottlieb et al., “The NYU Ultracomputer—Designing an MIMD shared memory parallel computer,” *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 175–189, Feb. 1983.
- [11] ARM Limited, “ARM Architecture Reference Manual ARMv8, for ARMv8-A architecture profile,” document DDI 0487B.a, 2017.
- [12] RISC-V Foundation, “The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2,” 2017.
- [13] ISO/IEC JTC1/SC22/WG21, “P3111R0: Atomic Reduction Operations,” 2024.
- [14] J. Lowe-Power et al., “The gem5 Simulator: Version 20.0+,” *arXiv:2007.03152*, 2020.
- [15] Y. Zhang et al., “Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs Through In-Cache Atomic Operations,” in *Proc. 57th Int. Symp. Microarchitecture (MICRO)*, 2024.
- [16] J. R. Goodman, M. K. Vernon, and P. J. Woest, “Efficient synchronization primitives for large-scale cache-coherent multiprocessors,” in *Proc. 3rd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 1989, pp. 64–75.
- [17] M. Sharma and Y. Alkhameri, “Cache coherence protocols on sparse-matrix vector multiplication (SpMV) and graphs in GEM5,” unpublished manuscript, 2026. [Online]. Available: <https://alkhameri.com/papers/cache-coherence-spmv-graphs-gem5.pdf>